Improved Simulation of Elastic Objects using 2DGS & Depth Information

Sotirios Karapiperis*
ETH Zürich
skarapiperis@ethz.ch

Hongjian Wang*
ETH Zürich
wanghon@ethz.ch

Adrian Denzler*
ETH Zürich
addenzler@ethz.ch

Abstract

Recent advancements in differentiable rendering have improved the task of geometry reconstruction from multiple views. However, these methods mainly focus on reconstructing static scenes with rigid objects ignoring the real nature of the objects. This work builds upon a recent method that focuses on reconstruction and simulation of elastic objects. More specifically, the dynamic system is modeled as a Mass-Spring system while the appearance and geometry is modeled using the Gaussian Splatting framework. Being differentiable, this simulation method allows the estimation of dynamic parameters by optimization in observation space. Combined with the differentiable rendering pipeline, this method achieves estimation of dynamic and appearance parameters using only photometric losses. In this work, we extend the aforementioned method, removing the assumptions made about the Gaussians. We further explore the use of alternative 2D primitives and the use of additional supervision signals. We present our results both on synthetic and real word dataset and draw useful observations about this line of work. ***Project Webpage:*** *https://zireekoon.github.io/3d_vision_project/*

1. Introduction

Reconstructing Scenes and Objects from multiple views has been a central task in Computer Vision and Graphics. Traditional approaches such as Structure from Motion[9] and Multiview Stereo produce sparse and dense reconstructions, respectively, while also estimating the camera poses. However, these approaches are computationally demanding, fail in texture-less areas, often produce floaters and cannot recover from errors.

Recently, advancements in differentiable rendering[6, 12] have facilitated the reconstruction of objects by differentiably rendering primitives and optimizing in image space comparing against ground truth images from multiple viewpoints. However, these methods mainly focus on statics scenes and rigid bodies.

However, reconstructing and simulating *deformable* objects from visual data is a vital problem, with wide-ranging applications in robotics, augmented reality, and physical modeling. In particular, predicting the behavior of elastic objects under deformation is essential for physically plausible simulations and future state prediction in interactive systems. Traditional methods for non-rigid reconstruction such as finite element methods (FEM) or Material Point Methods (MPM) have demonstrated strong capabilities in modeling realistic deformations. However, they often suffer from high computational complexity, require explicit meshing or material priors, and typically lack integration with appearance-based, photometric information. This lack of photometric supervision limits their effectiveness when dealing with raw image data or complex lighting conditions.

To address these limitations, the Spring-Gauss model [13] proposes an elegant integration of physical modeling with radiance field representations. It combines a mass-spring system to simulate elasticity with the differentiable rendering capabilities of 3D Gaussian Splatting (3DGS) [5]. This approach enables both geometric and photometric supervision, allowing accurate tracking of object deformation directly from visual observations. However, the Spring-Gauss framework makes simplifying assumptions: notably, it uses isotropic 3D Gaussians and ignores rotational alignment of primitives, which often leads to inaccurate appearance reconstruction and limited physical plausibility in dynamic scenes.

Moreover, while Gaussian Splatting enables differentiable and efficient rendering, its standard formulation was not designed with deformable physics in mind. When Gaussians are transformed via simulated dynamics, their coarse geometric abstraction can lead to misalignment between rendered appearance and actual object shape, particularly under non-rigid transformations.

In this work, we extend Spring-Gauss by removing the isotropic Gaussians assumption while introducing a separate supervision signal during fitting. Building on the Spring-Gauss framework, we propose several key improvements. First, we introduce rotational alignment of Gaus-

*All authors contributed equally to this work

sian primitives using the Kabsch algorithm to better preserve local geometry during deformation. Second, we replace isotropic 3D Gaussians with more geometrically accurate *2D Gaussian Splatting (2DGS)* primitives [2], improving the expressiveness and realism of rendered objects. Finally, we integrate depth-based regularization, leveraging synthetic depth ground truth from Blender to provide auxiliary supervision and enhance geometric fidelity.

Our extended system improves both photometric accuracy and physical realism, particularly in scenarios with complex deformations or significant rotations. This work further demonstrates the potential of combining physics-inspired modeling with modern differentiable rendering techniques to advance deformable object reconstruction.

2. Related Work

A combination of recent advancements in different fields made our work possible.

Static and Dynamic Gaussian Splatting. Kerbl et al. [5] initially proposed static 3D Gaussian Splatting, renowned for efficiently rendering static scenes using radiance fields. Dynamic extensions include works by Luiten et al. [7], who introduced dynamic persistent tracking, and Wu et al. [10], who proposed a unified 4D Gaussian Splatting framework, further advancing real-time rendering and dynamic modeling.

Mass-Spring Systems and Elastic Simulation. Zhong et al. [13] developed SpringGaus, pivotal in simulating elastic objects via mass-spring systems. Despite its success, the use of isotropic Gaussians limits capturing complex rotational and anisotropic behaviors, restricting broader applicability.

Geometric Improvements via Anisotropic Representations. Huang et al. [2] introduced 2D Gaussian splatting for geometrically accurate radiance fields. This approach significantly enhanced visual realism and geometric fidelity over traditional isotropic representations, laying groundwork for our similar principles.

Depth-informed Optimization Techniques. Chung et al. [1] developed methods integrating depth regularization into Gaussian splatting frameworks. Their approach optimizes 3D Gaussian models using sparse depth information, achieving notable improvements in geometric consistency and photometric accuracy. However, their method primarily targets sparse scenarios, prompting our extension to dense depth-informed simulations.

Physics-informed Dynamic Simulations. Xie et al. [11] advanced physics-integrated Gaussian splatting through PhysGaussian, embedding robust physical principles into generative dynamics. Similarly, Sharabi et al. [8] enhanced physical realism in simulations via the differentiable Neural Material Point Method (NeuralMPM), offering physics-informed particle-based modeling strategies.

Despite substantial advancements, literature highlights persistent challenges related to accurately modeling rotational dynamics, computational efficiency, and fully leveraging comprehensive depth and physical constraints. Our research directly addresses these gaps by combining rigorous geometric modeling with depth-informed optimization, achieving enhanced realism and accuracy in deformable object simulation.

3. Method

Our method builds upon the *Spring Gauss* method. In the next section, we briefly present Spring Gauss [13] and highlight the parts relevant to our work.

3.1. Spring Gauss

Spring Gauss combines 3D Gaussian Splatting with 3D Mass Spring systems to model the appearance, geometry and dynamics of elastic objects. The pipeline is shown in Fig. 1. More specifically, the optimization happens in two phases.

During the first phase, the images corresponding to the first frame are used to fit the gaussians. During this phase, only the Gaussian parameters are optimized. After convergence, a set of *anchor points* is sampled from the Gaussian means. These anchor points represent the masses of the *Mass Spring* system.

During the second phase, the *Mass Spring* system is simulated using the symplectic euler scheme where each anchor point is connected to its n_k nearest neighbors. While simulating, the positions of the anchor points are changing and so are the positions of the gaussians. More specifically, the position of each gaussian is defined by its n_b closest anchor points at the time of sampling using the Inverse Distance Weighting (IDW) interpolation technique. After each simulation step, the new positions of the anchor points are used to calculate the new positions of the Gaussians. During this state, both the *Mass Spring* parameters and the Gaussian parameters can be optimized. Since both the symplectic euler and IDW techniques are differentiable and, as previously mentioned, 3DGS is a differentiable rendering technique, the Mass Spring parameters are learned using only photometric losses that compare the rendered images with the ground truth images.

Since the aforementioned method does not update the rest of Gaussian parameters, and most notably the rotations of them, the authors choose to represent Guassians using an isotropic covariance matrix. Moreover, instead of using direction-depending texture features, they represent texture with a single RGB vector.

3.2. Handling the Rotations

We argue that restricting Gaussians to be isotropic and model perfectly diffuse objects is a strong assumption to

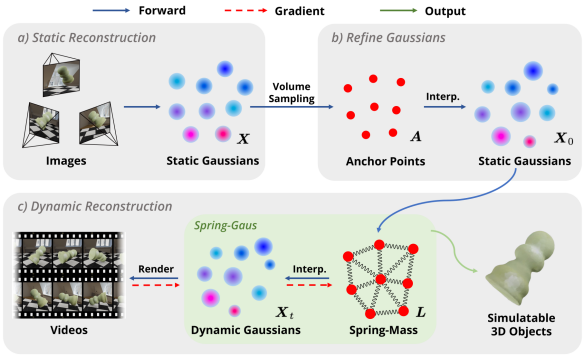


Figure 1. The pipeline of Spring Gauss. Image from [13].

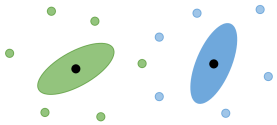


Figure 2. Visual Illustration of the use of Kabsch method in our case. Anchor points before and after the simulation step are used to calculate the optimal rotation matrix and applied to the Gaussian primitive.

be made. Moreover, this issue restricts us from also exploring the use of 2D primitives that by construction need to be rotated. To this end, we propose a simple way to update the rotation of the Gaussians during the simulation. More specifically, for each gaussian we have access to its n_b anchor points before and after the simulation step. For each point cloud we find the position of the Gaussian mean given by IDW interpolation and remove this vector from all the anchor points. Given these two *centered* point clouds we can find the best rotation matrix that aligns them using the Kabsch method [4], even though the real transformation may not be a rotation matrix. In turn, we can apply this rotation to the corresponding Gaussian. A concept figure is shown in Fig. 2

Given this modification, extending the original work to use anisotropic Gaussians is straightforward.

3.3. 2D Primitives

In our experiments we found that *Spring Gauss* is sensitive to anchor points locations and in turn sensitive to the result of fitting during the static phase. Given these observations, we explore the use of other primitives that may lead to geometrically more accurate fittings, e.g. the final positions of the gaussians are more closely related to the true geometry of the object.

2D Gaussian Splatting[2] is an alternative to 3DGS where the primitives are 2D Gaussians (e.g. ellipses instead

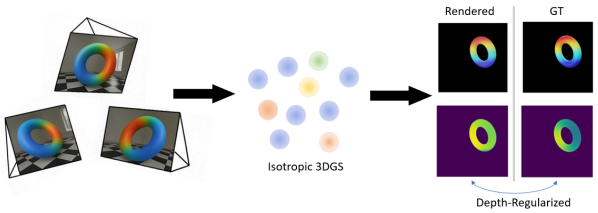


Figure 3. Overview of our depth-regularized 3D Gaussian Splatting framework. The rendered depth maps are aligned with simulated ground truth depth maps from Blender. A smoothness loss is additionally imposed across adjacent Gaussians to promote geometric consistency.

of ellipsoids). The main motivation behind this choice is the claim of the authors that these primitives results in geometrically more accurate fittings. The interface of 2DGS is very similar to the 3DGS interface and given our aforementioned modification, the integration is straightforward.

In accordance to the original work, we also incorporate the *Distortion* and *Normal Consistency* losses that force the 2D Gaussians to comply with the original geometry.

In our experiments we explore the use of both isotropic and non isotropic 2D Gaussians.

3.4. Depth-Regularized 3D Gaussian Splatting

Depth prior has been demonstrated to provide global geometric cues for neural point-based rendering, helping eliminate geometric artifacts and ambiguities [1]. Aligned with the goal of our work—to enhance the spatial coherence of isotropic 3D Gaussian representations—we incorporate depth supervision and geometry-aware regularization into the optimization process of 3D Gaussian Splatting (3DGS).

Since pre-trained monocular depth estimation models (e.g., MiDaS, DPT) perform poorly on the synthetic, segmented dataset used in our study, we instead simulate ground truth depth maps during rendering using Blender. These rendered depth maps serve as supervision signals to constrain the geometry of the splats.

To ensure consistency between predicted and target depth, we apply *inverse depth scaling* to normalize the rendered depth and improve numerical stability, especially in the foreground where fine structure is most critical. The depth loss is computed in image space by projecting the 3D Gaussians and comparing the rendered depth map D_{pred} with the simulated ground truth depth D_{gt}, using an ℓ_2 penalty after alignment.

In addition to depth supervision, we introduce a local smoothness regularization across neighboring Gaussians. Let d_i and d_j denote the depth values of adjacent splats. We define the smoothness loss as:

$$\mathcal{L}_{\text{smooth}} = \sum_{d_j \in \mathcal{N}(d_i)} \mathbf{1}_{[i \sim j]} \cdot \|d_i - d_j\|^2 \qquad (1)$$

Here, $\mathcal{N}(d_i)$ denotes the set of neighboring splats of d_i, and $\mathbf{1}_{[i\sim j]}$ is an indicator function that equals 1 if splats i and j are spatially adjacent, and 0 otherwise. This loss encourages smooth depth transitions between neighboring splats, helping reduce jitter and promoting a more coherent reconstructed surface.

The overall training loss is formulated as:

$$\begin{aligned}\mathcal{L} = (1-\lambda_{\text{ssim}})\mathcal{L}_{\text{color}} + \lambda_{\text{ssim}}\mathcal{L}_{\text{D-SSIM}} \\ +\lambda_{\text{depth}}\mathcal{L}_{\text{depth}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}}\end{aligned} \tag{2}$$

where $\mathcal{L}_{\text{color}}$ and $\mathcal{L}_{\text{D-SSIM}}$ guide the photometric appearance, while the added depth-related losses ensure geometric faithfulness and reduce rendering ambiguities.

4. Results

4.1. Dataset

4.1.1 Blender-Based Dataset Simulation with Depth Augmentation

To create a dataset suitable for studying visual and geometric perception, we adopt and extend a simulation and rendering framework originally proposed by Zong et al. [13]. The pipeline consists of two main stages: a physics-based simulation that generates deformable object motion, and a rendering stage that produces high-quality images along with corresponding depth information.

Simulation of Deformable Objects. The first stage involves simulating the physical behavior of soft or elastic materials under external forces. This is done using either a Material Point Method (MPM) or a spring-mass model, depending on the configuration. These simulations are temporally discretized, and at each time step, the geometry of the deforming object is recorded in the form of a mesh. The result is a sequence of 3D mesh files that represent the full dynamic behavior of the object over time.

Rendering Visual and Geometric Observations. Once the simulation has produced the sequence of 3D geometries, the second stage uses Blender's rendering engine to generate synthetic observations from these meshes. Each frame of the simulation is rendered from a fixed camera viewpoint. In addition to standard RGB images, our modified pipeline extracts depth maps for each frame. This is achieved by accessing Blender's internal Z-buffer and exporting it in a format suitable for machine learning applications.

Lighting conditions, material properties, and camera parameters are controlled to ensure consistency across frames and simulations. The rendering setup aims to reflect realistic visual conditions while maintaining physical coherence with the simulated geometry.

Enhancements and Contributions. Compared to the original framework [13], our version introduces the following improvements:

The rendering pipeline has been extended to include depth map extraction, enabling pixel-wise geometric supervision. The process is fully automated, rendering all simulation frames in sequence and exporting synchronized RGB and depth data.

The structure remains compatible with the original configuration system, making it easy to integrate with different simulation scenarios.

Real-world Dataset For the Real-world experiments we use the Real-world Dataset from [13]. Given that the static part needs dense viewpoints, the authors create the dataset in two stages. For the static part, they capture the scene from 70 viewpoints which they use to fit the gaussians. For the dynamic part, they use 3 cameras that concurrently capture the motion. Due to this discrepancy, the static gaussians are not on the same coordinate frame as the dynamic cameras. Thus, an additional *registration network* is employed that learns the *scale*, *rotation* and *translation* that align the two frames.

4.2. Metrics

We evaluate the quality of reconstruction using four complementary metrics: Peak Signal-to-Noise Ratio (PSNR), Structural Similarity Index (SSIM), Chamfer Distance (CD), and Earth Mover's Distance (EMD). PSNR and SSIM are widely used image quality metrics that quantify pixel-wise accuracy and perceptual similarity between the rendered and ground truth images.

In contrast, CD and EMD are 3D geometric metrics that measure the distance between the predicted and reference point clouds, capturing the fidelity of the reconstructed 3D geometry. Higher PSNR and SSIM indicate better image reconstruction quality, while lower CD and EMD values correspond to more accurate and compact geometric representations.

Similar with [13], during training, the model is simulated for the first 20 frames. During the evaluation procedure, we predict the next 10 frames. Having the gt images and geometry, we calculate the 4 metrics per frame for these 10 frames and calculate the mean value.

4.3. Quantitative Comparison

Simulated Dataset We evaluate our approach using both photometric and geometric metrics. For all our experiments we use the object *torus*. We find that there is variance between two identical runs, thus we present the average of two runs.

Table 1 summarizes the results across five model variants where I stands for Isotropic, A for Anisotropic, D for Depth supervison. Our 2DGS models outperform the 3DGS isotropic baseline in PSNR and SSIM, while achieving a significantly lower CD in the isotropic variant. Notably, the depth-regularized 3DGS model achieves the best EMD

Model	PSNR↑	SSIM↑	CD↓	EMD↓
3DGS I (baseline)[13]	14.89	0.896	3.282	0.069
3DGS A (ours)	**16.81**	**0.919**	4.223	0.095
2DGS I (ours)	15.86	0.908	2.716	0.091
2DGS A (ours)	14.98	0.899	5.940	0.114
3DGS D (ours)	15.47	0.881	3.038	**0.064**
2DGS I+D (ours)	16.54	0.918	**2.290**	0.081

Table 1. Quantitative comparison of rendering and geometry accuracy across models on simulated *Torus*. Higher PSNR and SSIM indicate better image reconstruction, while lower CD and EMD reflect improved 3D geometry. We highlight the 2 best model per metric. **I**: isotropic; **A**: Anisotropic; **D**: depth.

score, demonstrating its ability to generate compact and well-aligned geometric representations. Finally our model that combines both 2D Isotropic primitives and Depth-regularization, achieves the best CD results, the second best EMD and PSRN and almost identical with the best SSIM. Overall, this model performs better in terms of geometry while also achieving very good photometric results.

4.4. Visual Comparison

We present qualitative results for the synthetic dataset in Fig. 4 showcasing the GT images, the Baseline[13], the best performing model overall (2DGS I + D), the best in terms of PSNR and SSIM (3DGS A), the best in terms of CD (2DGS I) and the best in terms of EMD (3DGS D). We observe that both 3DGS A and 2DGS I + D perform visually better than the baseline, however all models diverge from the GT images as the simulation proceeds.

Since ground-truth depth is not available for the real-world dataset, we compare the 2DGS I and 3DGS A methods against the baseline. As shown in frame 8, 3DGS A exhibits slightly more realistic dynamics than 2DGS I and Baseline.

4.5. Discussion

In the next sections we present results validating or explaining our choices and discuss their effect.

Sensitivity to Initial Gaussians. As previously mentioned we notice that there is a discrepancy in the results between two identical runs. Investigating the source of this, reveals that the method is highly sensitive to the result of the static fitting. During static fitting, small numerical errors add up and produce sligtly different results for the location and the number of the Gaussians. This in turn leads to different anchor points and thus different velocity and dynamic parameters estimation results. In contrast, when running two identical experiments given the same pretrained static scene, the results of two simulations are similar. To this end, in all our experiments we run the same configuration two times and present the mean of the results.

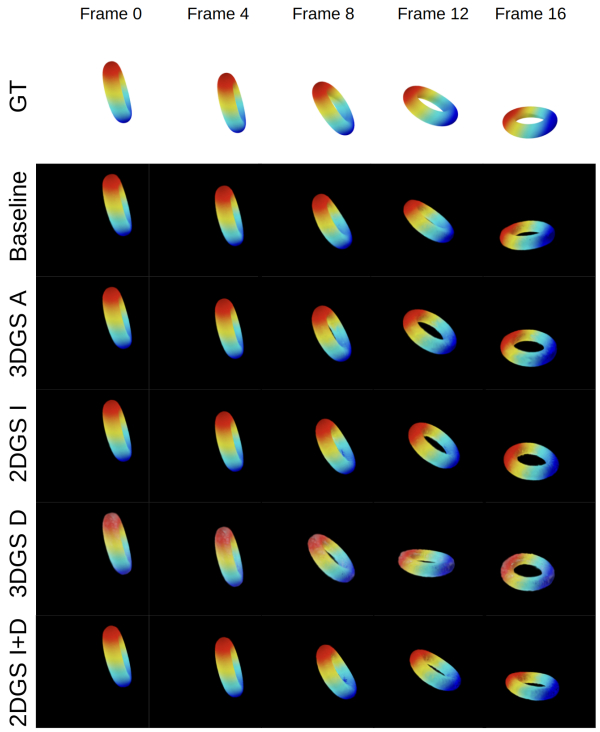


Figure 4. Visual comparison on the synthetic dataset.

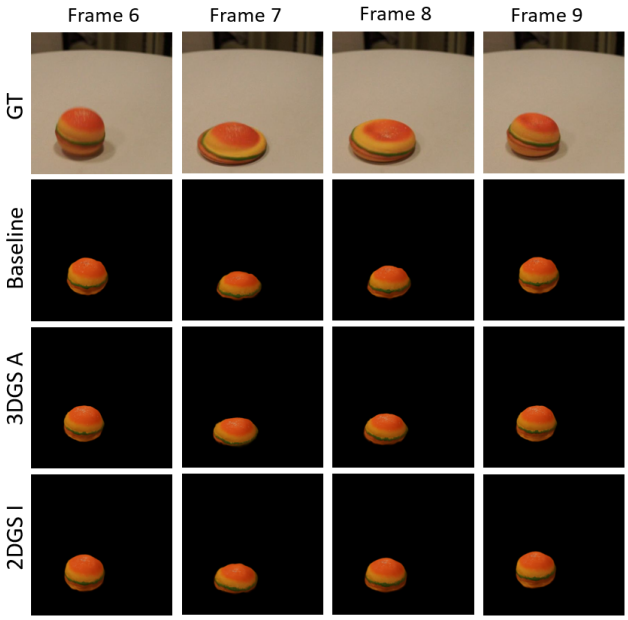


Figure 5. Visual comparison on the real dataset.

Kabsch Method. To validate that our proposed method has a visual impact, we rotate the Gaussian means with a known rotation matrix around their center of mass. Then, we compare the result of applying the same rotation matrix on the

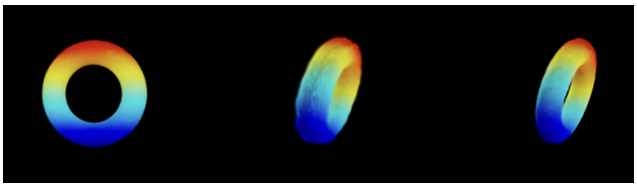


Figure 6. To validate our method we rotate the gaussian means with a known rotation matrix and apply or skip the same rotation to the Gaussians. We show the initial image (Left), the result of rotating only the Gaussian means (Middle) and finally the results of rotating both the Gaussian means and their Rotation matrices (Right).

Model	PSNR↑	SSIM↑	CD↓	EMD↓
2DGS I	15.86	0.908	**2.716**	**0.091**
2DGS I + losses	**16.27**	**0.915**	7.318	0.116

Table 2. 2DGS Regularization losses ablation results

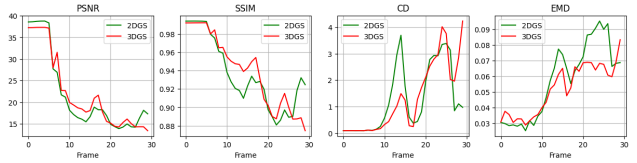


Figure 7. Per frame metrics show the performance of the simulation over time.

Gaussians or skipping this step. The results are shown in Fig. 6. Indeed, after rotating the Gaussians we get a visually more pleasant result.

2D Primitives. Moreover, we compare the effect of the additional losses on the performance. Counterintuitively, we find that integrating the *distortion loss* and *normal loss* produces better results in terms of appearance but worse results in terms of geometry. We present our results in Table 2 Moreover we observe that 2DGS primitives result in a higher number of points, producing 12k 2D Gaussians in comparison to 8k 3D Gaussians of the Baseline.

Metrics-Frame Graph. In Fig. 7 we present the per-frame metrics for the baseline and the best 2DGS I. As previously mentioned, the model during training observes only the first 20 frames. However, we observe that even in these seen frames, the model rapidly, after a few frames diverges from the ground truth data. More specifically, during the first 5 frames the object is in free-fall, there is no elastic deformation and hence the metrics indicate a good performance. At frame no. 6, the collision with the ground occurs, and the real interaction between the masses of the Mass-Spring system takes place. At this point, both models begin to diverge from the ground truth. The two spikes(downwards for PSNR and SSIM & upwards for CD and EMD), happening around frames 12 and 22, indicate that the response of the interaction with the ground differs from the expected response.

Depth-Regularized 3DGS. Previous works [1] that integrate depth into 3DGS typically employ a pre-trained monocular depth estimation model. However, the estimated depth maps suffer from scale ambiguity and must be explicitly aligned, for example, by optimizing the dense predicted depth to match the sparse but metrically correct SfM estimates. An alternative direct approach to obtaining depth maps is to use a depth sensor, such as an RGB-D camera. For instance, the recent work PhysTwin [3] leverages an RGB-D video dataset captured with multiple cameras, and employs a Chamfer distance-based optimization in the first stage of their multi-stage fitting process. However, when applying depth in real-world datasets, it is always necessary to perform segmentation to obtain a foreground mask—for example, to isolate dynamic objects before conducting further spring-mass modeling.

5. Conclusion

Conclusion. In summary, in this work we proposed a way to estimate the rotation of the Gaussian primitives attached to a Mass Spring system. In turn, that allowed us to extend a recent method to use both anisotropic and isotropic primitives. In addition, we replaced 3D Gaussians with more geometrical accurate 2D Gaussians and explored the benefits of using an additional depth supervision signal during the Gaussian fitting. Finally, we present a comparison between the different models and present useful observations about their performance.

Limitations. As previously mentioned, this method is highly sensitive to the result of the static fitting because this directly affects the position of the anchor points. A more sophisticated sampling procedure for the creation of the anchor points may aleviate this phenomenon. In addition, this work assumes perfectly diffuse objects. In contrast, the majority of the objects exhibit some kind of reflectance and thus Spherical Harmonics or related methods can be used in combination with our Kabsch-based method to model their appearance. Finally, we showcased the effect of Depth-regularization using ground truth depth on a synthetic dataset. This approach can be extended with depth predicted from pretrained model for use in Real World datasets.

6. Work Distribution

During this work, Sotirios was responsible for the Kabsch method integration and 2DGS Experiments, Hongjian was responsible for the Depth-regularization experiments and experiments on Real World data and Adrian was responsible for creating the synthetic dataset in Blender and producing the Project page.

References

[1] Jaeyoung Chung, Jeongtaek Oh, and Kyoung Mu Lee. Depth-regularized optimization for 3d gaussian splatting in few-shot images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, 2024. 2, 3, 6

[2] Yi-Hua Huang, Yang-Tian Sun, Ziyi Yang, Xiaoyang Lyu, Yan-Pei Cao, and Xiaojuan Qi. SC-GS: Sparse-controlled gaussian splatting for editable dynamic scenes. pages 4220–4230. Conference Name: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) ISBN: 9798350353006 Place: Seattle, WA, USA Publisher: IEEE. 2, 3

[3] Hanxiao Jiang, Hao-Yu Hsu, Kaifeng Zhang, Hsin-Ni Yu, Shenlong Wang, and Yunzhu Li. PhysTwin: Physics-informed reconstruction and simulation of deformable objects from videos. 6

[4] W. Kabsch. A solution for the best rotation to relate two sets of vectors. *Acta Crystallographica Section A*, 32(5):922–923, 1976. 3

[5] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. 1, 2

[6] Shichen Liu, Tianye Li, Weikai Chen, and Hao Li. Soft rasterizer: A differentiable renderer for image-based 3d reasoning. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 7708–7717, 2019. 1

[7] Jonathon Luiten, Georgios Kopanas, Bastian Leibe, and Deva Ramanan. Dynamic 3d gaussians: Tracking by persistent dynamic view synthesis. pages 800–809. Conference Name: 2024 International Conference on 3D Vision (3DV) ISBN: 9798350362459 Place: Davos, Switzerland Publisher: IEEE. 2

[8] Omer Rochman-Sharabi, Sacha Lewin, and Gilles Louppe. A neural material point method for particle-based emulation. *Transactions on Machine Learning Research*, 2025. 2

[9] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4104–4113, 2016. 1

[10] Guanjun Wu, Taoran Yi, Jiemin Fang, Lingxi Xie, Xiaopeng Zhang, Wei Wei, Wenyu Liu, Qi Tian, and Xinggang Wang. 4d gaussian splatting for real-time dynamic scene rendering. pages 20310–20320. Conference Name: 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) ISBN: 9798350353006 Place: Seattle, WA, USA Publisher: IEEE. 2

[11] Tianyi Xie, Zeshun Zong, Yuxing Qiu, Xuan Li, Yutao Feng, Yin Yang, and Chenfanfu Jiang. PhysGaussian: Physics-integrated 3d gaussians for generative dynamics. 2

[12] Wang Yifan, Felice Serena, Shihao Wu, Cengiz Öztireli, and Olga Sorkine-Hornung. Differentiable surface splatting for point-based geometry processing. *ACM Transactions On Graphics (TOG)*, 38(6):1–14, 2019. 1

[13] Licheng Zhong, Hong-Xing Yu, Jiajun Wu, and Yunzhu Li. Reconstruction and simulation of elastic objects with spring-mass 3d gaussians. arXiv. Version Number: 3. 1, 2, 3, 4, 5